University of Bamberg

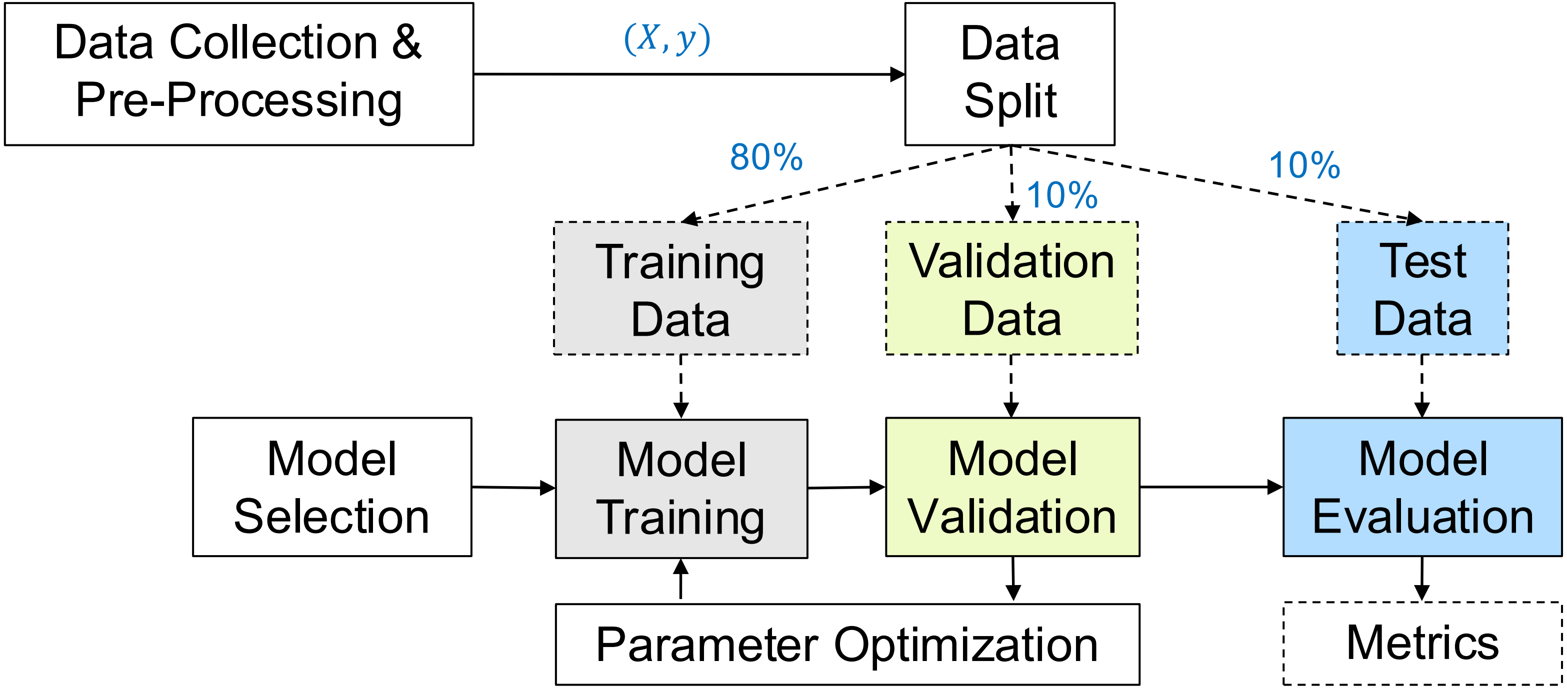
# Master Project - Machine Listening
## (CH-Proj-M)
## 05 – Project Pipeline

Prof. Dr.-Ing. Jakob Abeßer
Professur für Computational Humanities
Fakultät für Wirtschaftsinformatik und Angewandte Informatik
Otto-Friedrich-Universität Bamberg, 96047 Bamberg
E-Mail: jakob.abesser@uni-bamberg.de
Web: https://www.uni-bamberg.de/ch/

# Lecture
## Structure

| | | | |
|---|---|---|---|
| 6 | 18.5. **(P2)** | **3. Baseline system** (Audio analysis + machine learning method) | Basics: time-frequency representations & audio features |
| ~~7~~ | ~~25.05.~~ (Holiday) | | - |
| 8 | 01.06. | | Basics: machine learning & evaluation strategies & metrics |
| 9 | 08.06. | **4. Advanced System** (Extension using deep learning method) | Basics: deep learning |
| 10 | 15.06. | | tbd. |
| 11 | 22.06. | | tbd. |
| 12 | 29.06. | **Possible field trip (Fraunhofer IDMT)** | tbd. |
| 13 | 06.07. | Completion of term paper & presentation | **Deadline paper submission (6.7., EOB)** |
| 14 | 13.07. **(P3)** | **5. Project presentation** | |

# Machine Learning Pipeline
## Overview

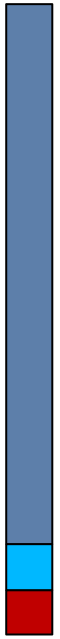



B9

# Machine Learning Pipeline

## Data Split

- Training data
  - Models learns from this data
- Validation / Development Set
  - Fine-tune the model (hyper)parameters
  - Model occasionally sees but does not learn from this data
- Test set
  - Only used once after model training
  - Should reflect the targeted real-world use case



# Machine Learning Pipeline
## Data Collection

- Check for available data resources for given task
- Collect, record, and annotate new data
- Ensure data variability

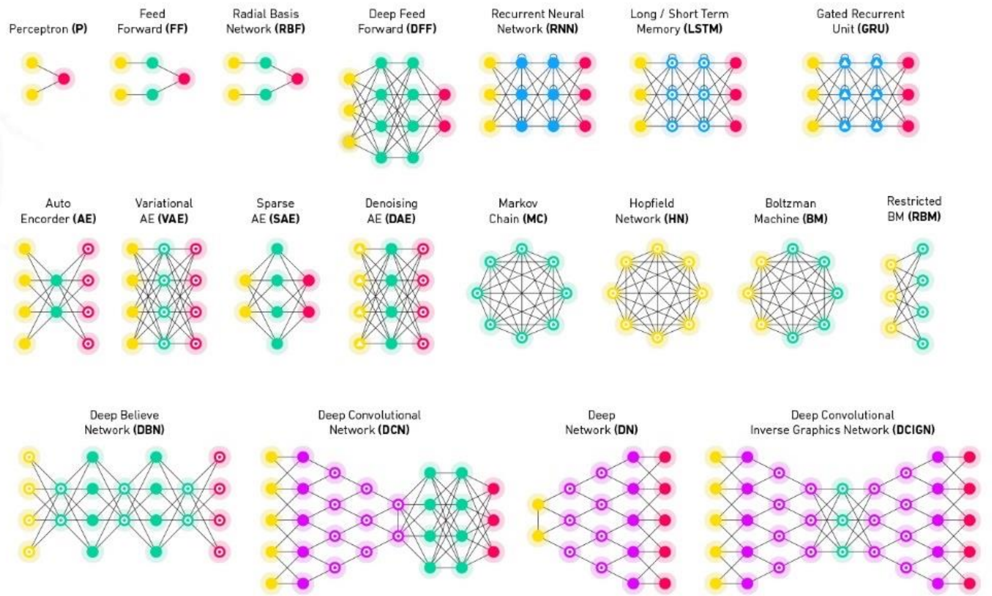
- Example (from acoustic condition monitoring)
  - Should include different motor engine types & conditions, recording locations, microphones, …

# Machine Learning Pipeline
## Data Cleanup & Pre-Processing

- Remove errors, silence, empty files, …
- Balance dataset (proportions among class examples)
- Normalize (depends on the model)

# Machine Learning Pipeline
## Data Cleanup & Pre-Processing

- Many model types (SVM, GMM, logistic regression, DNNs)
- Hyperparameters (SVM kernel functions, DNN layer types)




B10

# Machine Learning Pipeline

## Data Cleanup & Pre-Processing

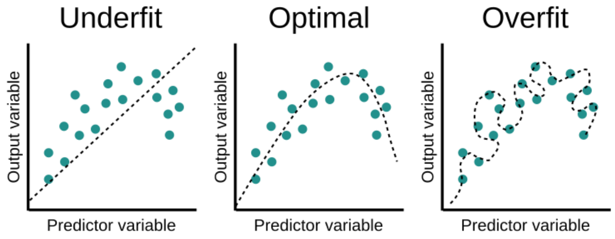
- Often constrained by the use-case / task
  - Model complexity (memory, training time, training data amount)
- Feature pre-processing depends on model type
- Use simple models for simple tasks

# Machine Learning Pipeline
## Model Training

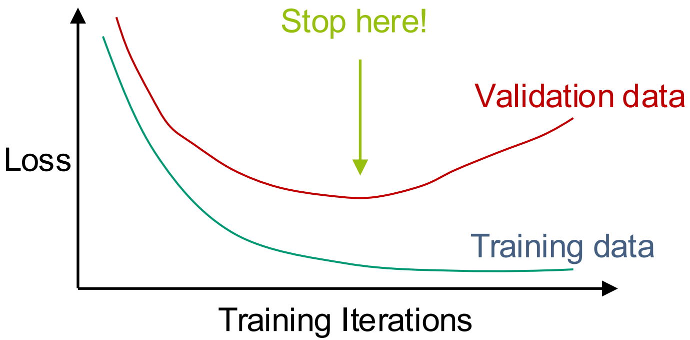
- Iterative process (learn from examples)
- (Random) parameter initialization
- Use (batches of) training data to iteratively improve model predictions (optimization)
- Update model parameters according to loss function
- Monitor improved performance
- Repeat until convergence

# Machine Learning Pipeline
## Model Validation

- Regular model evaluation each or multiple training iteration
  - Optimize model (hyper)parameters
  - Detect overfitting on training data
  - Stop the training




B11




B12

# Machine Learning Pipeline

## Model Evaluation (Example: Binary Classification)

- True/false positives (TP/FP)
- True/false negatives (TN/FN)
- Metrics
  - Precision
  - Recall
  - Accuracy
  - F-score

| | Prediction 1 | Prediction 0 |
|---|---|---|
| **Annotation 1** | **TP** *true positives* | **FN** *false negatives* |
| **Annotation 0** | **FP** *false positives* | **TN** *true negatives* |

True Positive Rate
Sensitivity
Recall

$$\mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP+FN}}$$

False Positive Rate

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP+FN}}$$

Specificity

$$\mathrm{Specificity} = \frac{\mathrm{TN}}{\mathrm{FP+FN}}$$

Precision

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP+FP}}$$

Accuracy

$$\mathrm{ACC} = \frac{\mathrm{TP+TN}}{\mathrm{TP+TN+FP+FN}}$$

B13

# Machine Learning Pipeline
## Baseline

- Motivation
  - How complicated is the task really?
    - This will not be your best system
  - This sets the “baseline” for your improved system (deep learning) to compare against

# Machine Learning Pipeline
## Baseline

- Options
  - Use available implementation of reference paper
  - Set up simple ML classifier <- Goal!
    - Use established audio features (e.g. MFCC)
    - Use traditional ML method (e.g. SVM, Random Forest)
  - ”Lazy” version
    - Use results from reference paper
    - Important: make sure experimental conditions are perfectly equal (dataset split, evaluation metric etc.)

# Image Sources

B9: © Jakob Abeßer

B10: Neural Network Zoo (https://www.asimovinstitute.org/wp-content/uploads/2019/04/NeuralNetworkZoo20042019.png)

B11: https://www.educative.io/api/edpresso/shot/6668977167138816/image/5033807687188480

B12: © Jakob Abeßer

B13: (Virtanen, et al. 2018), p. 170, Fig. 6.7

# Thank you for your attention!
# Any questions?

Prof. Dr. Jakob Abeßer

jakob.abesser@uni-bamberg.de

https://www.uni-bamberg.de/ch/